REFRAG: Rethinking RAG-based Decoding for LLM Efficiency

Xiaoqiang Lin, Aritra Ghosh, Bryan Kian Hsiang Low,
Anshumali Shrivastava, Vijai Mohan

딥러닝 논문 읽기
자연어처리팀

요약

RAG context에 발생하는 sparsity inherent와 unique structure를 활용해, memory 사용과 computational overhead를 줄이는 방법

- 효율적인 디코딩 프레임 워크
  - LLaMA-2-7B 기준, 최대 31배 더 빠르게 처리(이전 SOTA 연구였던 CEPE 대비 3.75배)
  - Perplexity 기준 성능 손실 없음

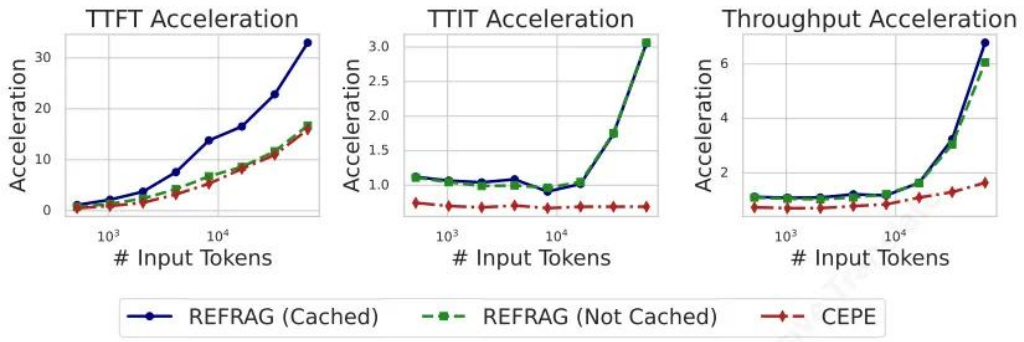


Figure 2 Empirical verification of inference acceleration of REFRAG with $k = 16$.

- 16배 더 많은 데이터를 한 번에 처리
- 다양한 long-context tasks에서 활용 (RAG, multi-turn conversations, long document summarization 등)

배경

- LLM의 성능은 좋아졌지만, long-context 처리는 Self-Attention구조로 인해 latency 증가
  - TTFT 지연은 제곱으로 증가
  - TTIT는 선형적으로 증가

- key-value cache로 인한 메모리 사용량 증가하고 throughput이 감소
  - KV cache에 대한 추가 메모리는 선형적으로 증가

- knowledge enrichment와 시스템 효율간의 트레이드 오프가 발생 (Liu et al., 2025)

- extensive context에 대한 추론 latency 최적화 연구들이 있지만, RAG-based applications 특징 반영 미흡
  - modifying the attention mechanism's complexity (Beltagy et al., 2020)
  - sparsifying attention and context (Child et al., 2019; Xiao et al., 2024; Jiang et al., 2024)
  - altering context feeding strategies (Yen et al., 2024)

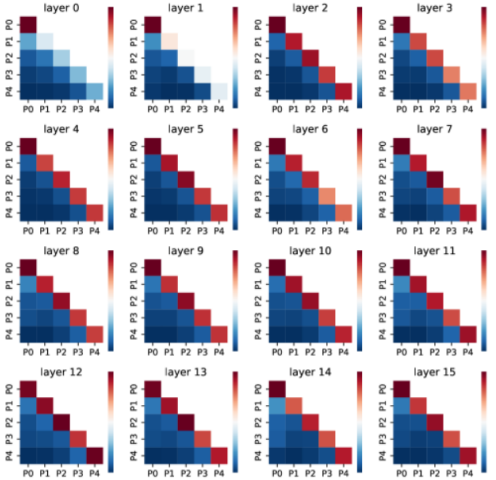
Contributions

Long Context에서의 TTFT 감소

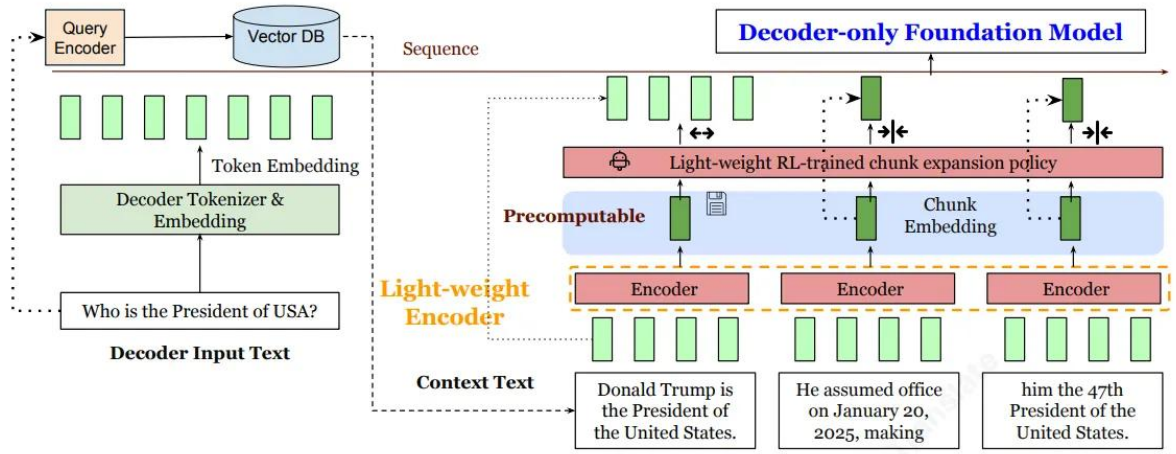
- Inefficient Token Allocation의 개선
  - 검색된 구절 중 대부분은 최종 답변에 거의 기여하지 않거나 유익하지 않음
  - 컨텍스트 입력이 길더라도 실제로 답변 생성에 필요한 핵심 정보는 작은 부분집합에 불과

- Wasteful Encoding and Other Information
  - 이미 수행된 많은 계산 결과가 디코딩 단계에서 버려지지 않고 재활용함
  - chunk 분할, chunk의 encoding(벡터)를 재사용

- Unusually Structured and Sparse Attention
  - 디코딩 과정에서 대부분의 context의 chunk들 사이에는 관련성이 없음
  - 주로 zero에 가까운 교차 cross-attention 패턴이 발생



방법론



- Continual pre-training
  - Reconstruction Task(+Curriculum learning)
  - Next token prediction(+Curriculum learning)
  - Mixed input pretraining
  - Selective compression via RL

- Fine-tuning on downstream tasks

방법론

- Reconstruction Task(+Curriculum learning)
  - Freeze된 decoder가 이해할 수 있는 벡터를 출력하도록 encoder를 학습
    - Decoder를 freeze
    - Context $x_{1:s}$를 encoder에 입력(s는 context의 마지막 token index)
    - Encoder의 결과를 Projection layer에 입력
      - Projection layer는 2-layer MLP
      - Proejction layer의 차원은 decoder의 Token embedding의 차원과 동일하도록 구성
    - Projection layer의 결과를 decoder가 복원하도록 학습(Teacher forcing 적용)
      - 고정된 chunk 길이를 설정
      - chunk 개수를 점진적으로 늘려가면서 복원

Factor	Stage 1	Stage 2	Stage 3	Stage 4	Stage 5	Stage 6	Stage 7	Stage 8	Stage 9	Summation
1×8	1333	445	148	49	16	6	2	1	0	2000
2×8	333	298	267	238	213	191	171	153	137	2000
4×8	83	102	126	156	193	238	293	362	447	2000
8×8	20	35	61	106	185	324	565	985	1719	4000
16×8	5	11	23	48	103	220	468	997	2125	4000
32×8	1	3	7	19	50	133	353	939	2496	4000
64×8	1	3	9	25	73	212	618	1802	5259	8000
128×8	1	3	9	25	73	212	618	1802	5259	8000
256×8	1	3	9	25	73	212	618	1802	5259	8000

Table 8 The geometry curriculum learning scheduling. The whole training is split into 9 stages. In each stage, we have a combination of different data (e.g., 1X8 means reconstructing 8 tokens, 2X8 means reconstructing 16 tokens). For each type of data, the number of samples in each stage is determined by a geometric sequence which sums up to the total number of samples in the last column. As training proceeds, the data mixture has more and more longer sequences.

방법론

- Next-token prediction(+Curriculum learning)
  - Decoder를 unfreeze하고, 전체 파라미터(encoder, decoder, projection) 학습
    - Context인 $x_{1:s}$를 encoder에 입력(s는 context의 마지막 token index)
    - Encoder의 결과를 Projection layer에 입력
      - Projection layer는 2-layer MLP
      - Projection layer의 차원은 decoder의 Token embedding의 차원과 동일하도록 구성
    - Projection layer의 결과를 decoder에 입력
    - Decoder가 $x_{s+1:o}$를 복원하도록 학습(Teacher forcing 적용)
      - 고정된 chunk 길이를 설정
      - Encoder에 입력하는 chunk 개수를 점진적으로 늘려가면서 복원

Factor	Stage 1	Stage 2	Stage 3	Stage 4	Stage 5	Stage 6	Stage 7	Stage 8	Stage 9	Summation
1×8	1333	445	148	49	16	6	2	1	0	2000
2×8	333	298	267	238	213	191	171	153	137	2000
4×8	83	102	126	156	193	238	293	362	447	2000
8×8	20	35	61	106	185	324	565	985	1719	4000
16×8	5	11	23	48	103	220	468	997	2125	4000
32×8	1	3	7	19	50	133	353	939	2496	4000
64×8	1	3	9	25	73	212	618	1802	5259	8000
128×8	1	3	9	25	73	212	618	1802	5259	8000
256×8	1	3	9	25	73	212	618	1802	5259	8000

Table 8 The geometry curriculum learning scheduling. The whole training is split into 9 stages. In each stage, we have a combination of different data (e.g., 1X8 means reconstructing 8 tokens, 2X8 means reconstructing 16 tokens). For each type of data, the number of samples in each stage is determined by a geometric sequence which sums up to the total number of samples in the last column. As training proceeds, the data mixture has more and more longer sequences.

방법론

- Selective compression via RL
  - Mixed-Input Pretraining
  - Policy network의 logit으로 softmax 계산하여, uncompression chunk 샘플링

$$\pi_\theta(l_t = i \mid x, \{l_j\}_{j=1}^{t-1}) := \pi_\theta(l_t = i \mid \{\mathbf{c}_j\}_{j=1}^{L}, \{l_j\}_{j=1}^{t-1}) = \frac{\exp(\mathbf{s}_i - \mathrm{n}_i)}{\sum_{j=1}^{L} \exp(\mathbf{s}_j - \mathrm{n}_j)}$$

  - p는 압축률을 고려하여 선정(결과적으로 16이 최적)

$$\text{compression rate} = \frac{k}{1 - p + kp}$$

  - GRPO 학습 적용
    - Sampling G trajectories

$$r_i = r\left(x, \{l_j^{(i)}\}_{j=1}^{T'}\right) = -\mathcal{M}_{\text{dec}}\left(x_{s+1:s+o} \mid E(x, \{l_j^{(i)}\}_{j=1}^{T'})\right) \qquad A_t^{(i)} = \frac{r_i - \text{mean}\left(\{r_i\}_{i=1}^{G}\right)}{\text{std}\left(\{r_i\}_{i=1}^{G}\right)} .$$

    - Update Policy network

$$\mathcal{J}_\theta = \frac{1}{G}\sum_{i=1}^{G} \mathbb{E}_{\substack{x \sim P(\mathcal{X}), \\ \{l^{(i)}\}_{i=1}^{G} \sim \pi_\theta([L] \mid x)}} \frac{1}{T'}\sum_{t=1}^{T'} \min\left[\frac{\pi_\theta(l_t^{(i)} \mid x, \{l_j^{(i)}\}_{j=1}^{t-1})}{\pi_{\theta_{\text{old}}}(l_t^{(i)} \mid x, \{l_j^{(i)}\}_{j=1}^{t-1})} A_t^{(i)}, \text{clip}\left(\frac{\pi_\theta(l_t^{(i)} \mid x, \{l_j^{(i)}\}_{j=1}^{t-1})}{\pi_{\theta_{\text{old}}}(l_t^{(i)} \mid x, \{l_j^{(i)}\}_{j=1}^{t-1})}, 1-\epsilon, 1+\epsilon\right) A_t^{(i)}\right]$$

실험

- Training Datasets
  - SlimPajama 내 long-context 데이터인 Book과 Arxiv를 1:1 비율로 구성하여 20B 데이터 확보
- Evaluation Datasets
  - Training Datasets에서 사전에 분리한 데이터
  - 모델의 Generalization 성능을 확인하기 위해 PG19와 Proof-pile datasets 사용
- Model
  - LLaMA-2-7B를 LLM으로한 REFRAG(본 연구), CEPE, REPLUG 방법과 비교
  - LLaMA-2-7B의 응용 모델도 추가
- Metric
  - 정확도: Perplexity
  - 속도: TTFT, TTIT, Throughput

실험

Table 1 Perplexity on output tokens $x_{s+1:s+o}$ given context tokens $x_{1:s}$ for different models. We use $s = 2048$ and $o \in \{512, 1024, 2048\}$ here. Bolding are based on comparing baselines excluding LLaMA-Full Context and LLaMA-32K since they are expected to be the best (ideally). The lower the better (↓).

	Arxiv			Book			PG19			ProofPile		
	P512	P1024	P2048	P512	P1024	P2048	P512	P1024	P2048	P512	P1024	P2048 ↓
LLaMA-Full Context	1.075	1.074	1.069	1.830	1.827	1.826	1.947	1.941	1.935	0.952	0.940	0.931
LLaMA-32K	1.086	1.084	1.076	1.887	1.883	1.880	1.988	1.982	1.975	0.961	0.948	0.937
LLaMA-No Context	1.526	1.371	1.254	2.101	1.995	1.927	2.211	2.102	2.030	1.437	1.256	1.127
LLaMA_{256}	1.267	1.221	1.171	**1.924**	1.897	1.874	2.031	2.003	1.978	1.156	1.094	1.038
REPLUG	1.526	1.371	1.254	2.101	1.995	1.927	2.211	2.102	2.030	1.437	1.256	1.127
CEPE	1.196	1.148	1.107	1.946	1.896	1.864	2.057	2.002	1.964	1.075	1.014	0.968
REFRAG_8	**1.124**	**1.091**	**1.062**	**1.905**	**1.868**	**1.844**	**1.996**	**1.956**	**1.927**	**0.997**	**0.952**	**0.916**
REFRAG_{16}	**1.157**	**1.114**	**1.076**	1.925	**1.882**	**1.853**	**2.016**	**1.971**	**1.938**	**1.034**	**0.976**	**0.931**
REFRAG_{32}	1.215	1.154	**1.103**	1.946	**1.896**	**1.862**	2.039	**1.987**	**1.949**	1.097	1.020	**0.961**

실험

$$\text{compression rate} = \frac{k}{1 - p + kp}$$

Table 2 Perplexity on output tokens $x_{s+1:s+o}$ given different length of context. We use $s \in \{4096, 8192, 16384\}$ and $o = 2048$ here. Bolding are based on comparing baselines excluding LLAMA-FULL CONTEXT and LLAMA-32K since they are expected to be the best (ideally). The lower the better (↓).

	Context Length =4096				Context Length=8192				Context Length=16384			
	Arxiv	Book	PG19	ProofPile	Arxiv	Book	PG19	ProofPile	Arxiv	Book	PG19	ProofPile ↓
LLAMA-FULL CONTEXT	6.751	6.956	6.829	6.701	9.675	9.069	8.963	9.401	9.043	8.913	8.848	8.989
LLAMA-32K	1.037	1.862	1.960	0.898	0.965	1.867	1.947	0.834	0.865	1.840	1.943	0.770
LLAMA-NO CONTEXT	1.253	1.925	2.030	1.126	1.226	1.949	2.032	1.110	1.174	1.939	2.041	1.081
REPLUG	1.253	1.925	2.030	1.126	1.226	1.949	2.032	1.110	1.174	1.939	2.041	1.081
CEPE	1.085	1.856	1.959	0.945	1.032	1.878	1.958	0.904	0.960	1.864	1.966	0.863
REFRAG_8	**1.042**	**1.837**	**1.922**	**0.894**	**0.983**	**1.839**	**1.922**	**0.858**	0.977	**1.840**	**1.939**	0.891
REFRAG_{16}	**1.058**	**1.847**	**1.934**	**0.910**	**0.994**	**1.845**	**1.932**	**0.871**	**0.942**	**1.840**	**1.945**	**0.850**
REFRAG_{32}	1.088	1.857	**1.946**	**0.944**	**1.032**	**1.860**	**1.945**	0.912	0.969	**1.852**	**1.955**	0.880

실험

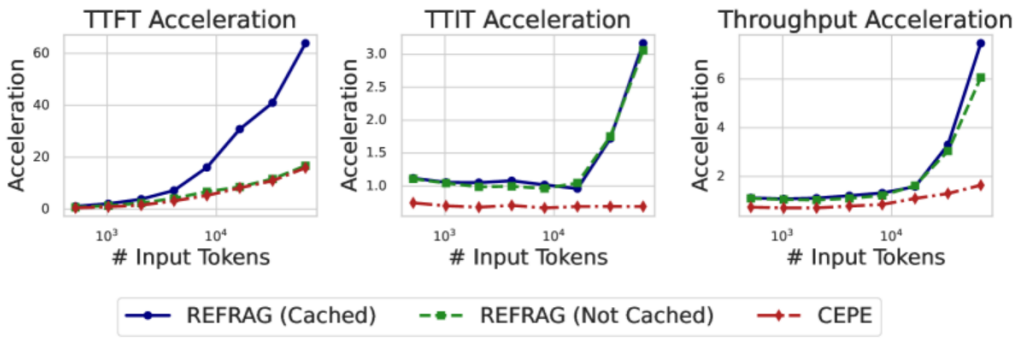


Figure 8: Empirical verification of inference acceleration of REFRAG with $k = 32$.

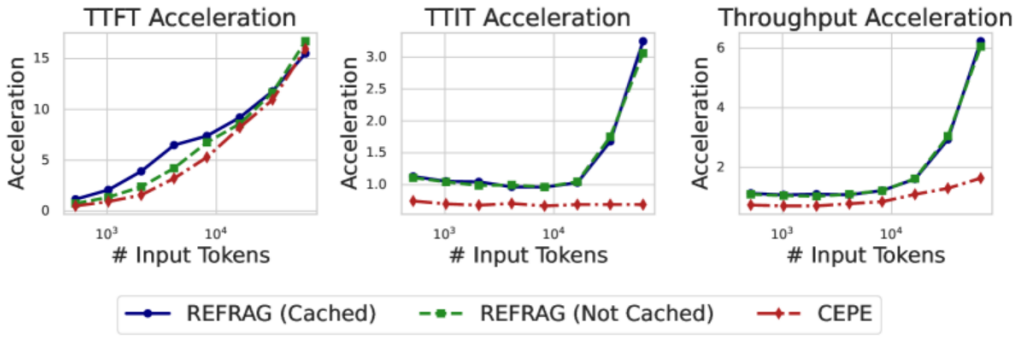


Figure 9: Empirical verification of inference acceleration of REFRAG with $k = 8$.

실험

$$\text{compression rate} = \frac{k}{1 - p + kp}$$

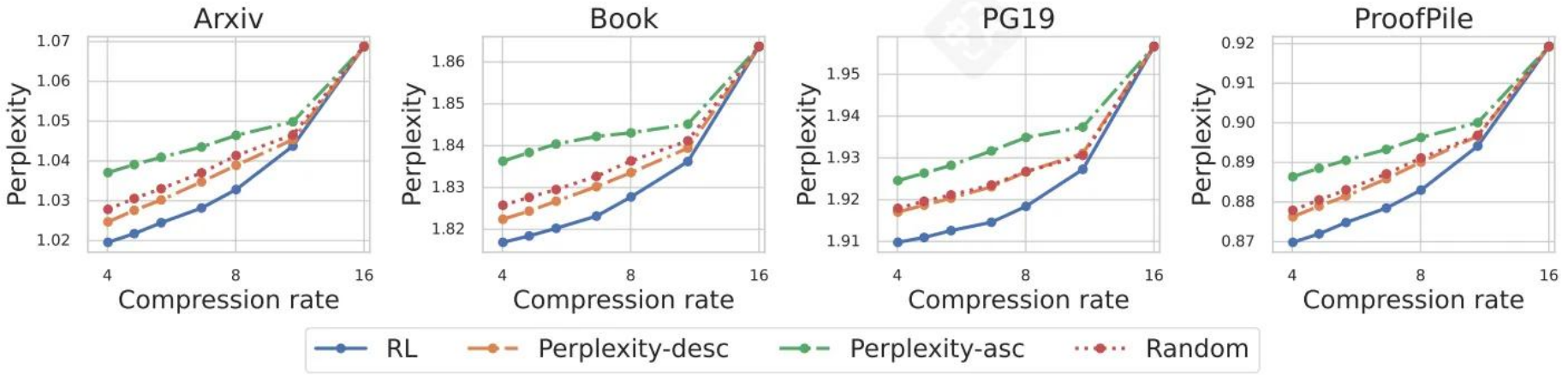


Figure 3 Perplexity on $x_{s+1:s+o}$ under varying compression rates by selectively compressing different percentages of chunks. We compare three selection methods: **RL** (trained policy), **Perplexity-desc** (heuristic: lower perplexity), **Perplexity-asc** (heuristic: higher perplexity), and **Random** (random selection).

Ablation Study

- Curriculum learning의 필요성

Table 11 Performance comparison on reconstruction task with and w/o curriculum learning. Perplexity is reported as average of Arxiv and Book domain.

	P16	P32	P128	P2048 ↓
LLaMA-Full Context	1.397	0.734	0.203	0.021
LLaMA-No Context	3.483	2.981	2.249	1.590
REFRAG w/o curriculum	3.719	3.098	2.272	1.599
REFRAG with curriculum	0.669	0.451	0.230	0.135

- Reconstruction Task의 필요성

Table 12 Performance comparison on continual pre-training task with and w/o continued from reconstruction task. Perplexity is reported as average of Arxiv and Book domain.

	P16	P32	P128	P2048 ↓
LLaMA-Full Context	1.448	1.458	1.464	1.449
LLaMA-No Context	3.483	2.981	2.249	1.590
REFRAG w/o reconstruction	3.272	2.789	2.119	1.544
REFRAG with reconstruction	2.017	1.837	1.632	1.453

- Selective Compression의 효과

Table 13 The performance of REFRAG under the same compression rate with full compression (i.e., REFRAG_8) and selective compression (i.e., $\text{REFRAG}_{16+\text{RL}}$).

		Arxiv			Book			PG19			ProofPile		
	Compression Rate	P512	P1024	P2048	P512	P1024	P2048	P512	P1024	P2048	P512	P1024	P2048 ↓
Context Length=2048													
REFRAG_8	8	1.124	1.091	1.062	1.905	1.868	1.844	1.996	1.956	**1.927**	0.997	0.952	0.916
$\text{REFRAG}_{16+\text{RL}}$	8.258	**1.118**	**1.090**	1.062	**1.878**	**1.856**	**1.840**	**1.978**	**1.952**	1.930	**0.992**	**0.951**	0.916
Context Length=4096													
REFRAG_8	8	1.098	1.065	1.042	1.895	1.860	1.837	1.989	1.950	1.922	0.965	0.923	0.894
$\text{REFRAG}_{16+\text{RL}}$	8.0157	**1.065**	**1.048**	**1.033**	**1.851**	**1.837**	**1.828**	**1.952**	**1.934**	**1.918**	**0.932**	**0.905**	**0.883**

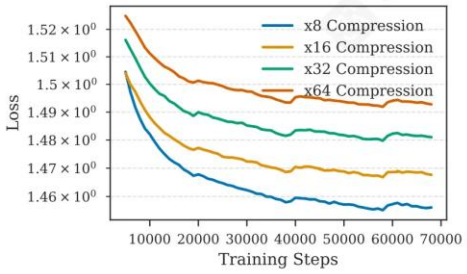


Figure 10 Training trajectory for our model with different compression rate.

Ablation Study

- encoder(LLaMA-2-7B, LLaMA-2-13B)와 decoder(ReBERTa-Base, RoBERTa-Large) 조합

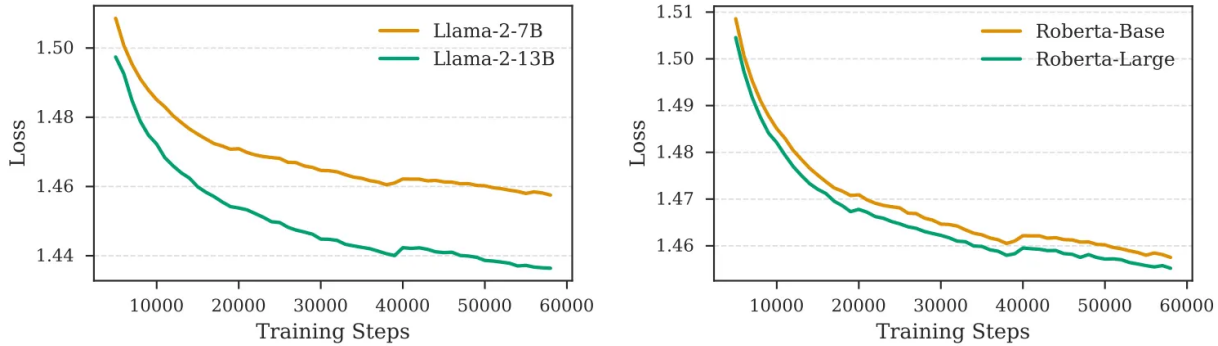


Figure 11 Training trajectory for different encoder and decoder combinations. On the left, we have two different decoder the Roberta-Base encoder. On the right we have two different encoder for LLaMA-2-7B decoder model.

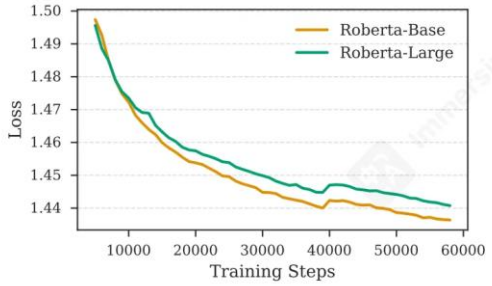


Figure 12 Training trajectory for different encoder paired with LLaMA-2-13B decoder.

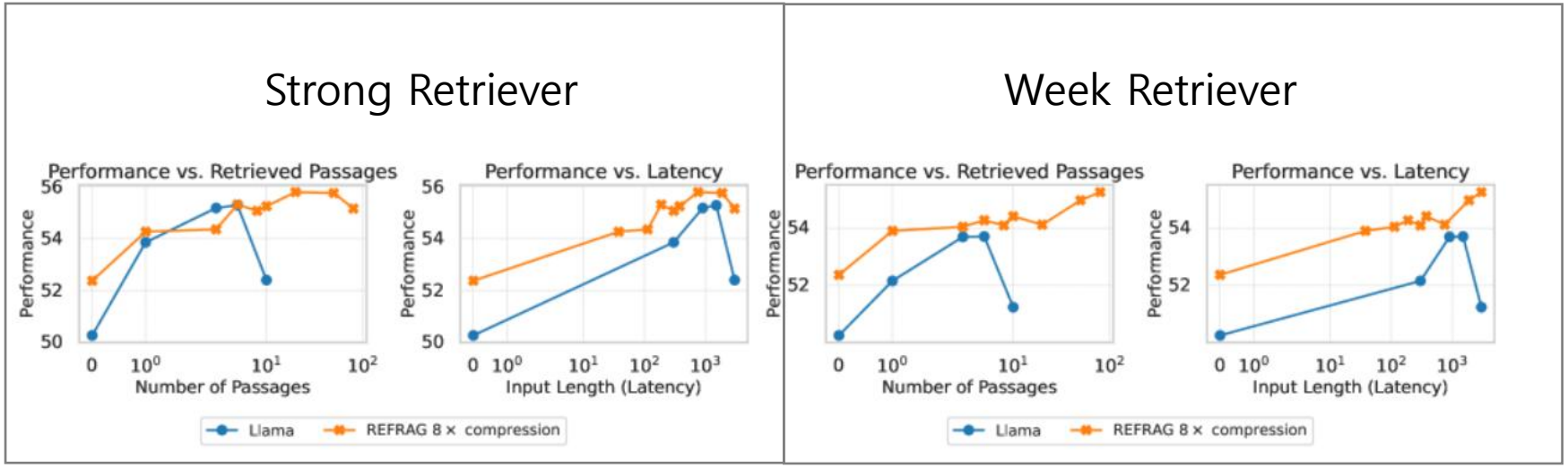
Application

- **RAG**
  - Train에는 5개 도메인의 QA 데이터셋 조합을 사용(110만개 데이터 포인트)
    - Dialogue(OpenAssistant Conversations Datset), Open-Domain QA(MathQA, Web, Wiki, Yahoo, Freebase, MS MARCO), Reading Comprehension(PubMedQA, SQuADv2 등), Chain-of-thought Reasoning(AlgebraQA 등)

  - Eval는 Train 데이터에 5%를 사용하고, 추가로 RAG에 일반적으로 사용되는 데이터셋을 추가함
    - MMLU, BoolQ, SIQA, PIQA, KILT
    - retrieval corpus로는 Wikipedia dumps와 CommonCrawl dumps를 사용
    - Strong Retriever, Weak Retriever를 구성하여 EM, ACC, F1으로 평가
      - Retriver로 DRAGON+ 사용

Application

- **RAG**



Application

- **RAG**

Table 3 Comparison of model performance of different models with different number of retrieved passages for RAG under the strong retriever scenario.

Generation	NQ	FEVER	TQA	WebQA	FreebaseQA	GSM8K	StrategyQA	BoolQ ↑	(1/ # tokens)
Short context with the same latency									
LLAMA$_{FT}$ + 1 passage	**23.96**	62.04	9.64	37.33	**75.18**	7.38	64.44	**29.24**	1×
REFRAG$_8$+ 8 passages	22.96	**66.59**	**13.05**	**38.67**	73.46	7.38	**75.56**	3.30	1×
REFRAG$_{16}$+ 8 passages	22.94	**62.88**	**12.97**	**42.67**	71.50	**9.40**	**71.11**	5.87	2×
REFRAG$_{32}$+ 8 passages	22.11	**64.24**	**12.57**	**41.33**	71.74	**12.75**	**73.33**	1.99	4×
Long context									
LLAMA$_{FT}$ + 10 passages	**26.08**	65.44	9.68	**40.00**	**76.17**	6.71	68.89	30.00	1×
CEPED +80 passages	0.03	65.68	0.01	0.00	0.00	0.00	0.00	**57.80**	
REPLUG +80 passages	-	-	-	-	-	-	64.44	-	
LLAMA-32K +80 passages	1.24	0.14	0.52	10.67	9.83	0.00	0.00	0.03	
REFRAG$_8$ +80 passages	24.15	**68.83**	**13.06**	37.33	74.20	**7.38**	**71.11**	7.03	1×
REFRAG$_{16}$ +80 passages	23.30	**66.01**	**12.65**	38.67	75.43	**12.08**	**73.33**	12.23	2×
REFRAG$_{32}$ +80 passages	23.02	**68.48**	**12.14**	38.67	71.74	**9.40**	68.89	6.42	4×
Multi-Choice	MMLU	CommonsenseQA	MathQA	ECQA	HellaSwag	SIQA	PIQA	Winogrande ↑	
Short context with the same latency									
LLAMA$_{FT}$ + 1 context	**50.23**	85.05	99.50	84.77	41.80	68.12	67.36	55.64	1×
REFRAG$_8$ + 8 passages	50.29	**92.27**	**99.66**	**94.70**	**45.23**	**68.94**	**71.38**	**57.70**	1×
REFRAG$_{16}$ + 8 passages	49.84	**89.18**	**99.66**	**98.01**	39.33	**68.42**	**70.29**	**56.67**	2×
REFRAG$_{32}$ + 8 passages	49.51	**91.75**	**99.50**	**97.35**	**42.86**	**68.17**	**68.34**	**56.75**	4×
Long context									
LLAMA$_{FT}$ + 10 passages	48.66	82.99	68.46	84.11	41.77	67.45	68.01	53.91	1×
CEPED +80 passages	26.26	26.29	23.66	24.50	24.95	32.86	48.53	44.51	
REPLUG +80 passages	-	78.35	-	76.16	-	65.51	-	-	
LLAMA-32K +80 passages	22.21	16.49	19.80	16.56	23.76	24.16	34.17	48.86	
REFRAG$_8$ +80 passages	**50.42**	**92.27**	**99.66**	**97.35**	**44.61**	**68.22**	**69.37**	**57.54**	1×
REFRAG$_{16}$ +80 passages	**50.88**	**89.69**	**99.66**	**96.69**	38.50	**68.47**	**70.89**	**56.99**	2×
REFRAG$_{32}$ +80 passages	**49.77**	**90.72**	**99.50**	**98.01**	**43.37**	**68.47**	**69.04**	**56.99**	4×

- means the corresponding model has out-of-memory error.

Table 15 Performance of our model under compression rate of 16 with different number of retrieved passages in RAG under the strong retriever scenario.

# Passages	MMLU	NQ	FEVER	WebQA	FreebaseQA	CommonsenseQA	ECQA	StrategyQA	HellaSwag	SIQA	PIQA ↑
0	48.07	18.73	65.80	34.67	60.20	89.18	87.42	68.89	43.72	67.25	70.18
1	50.49	21.39	69.46	37.33	68.06	86.60	89.40	80.00	43.26	68.17	70.08
3	50.49	22.01	66.02	38.67	71.01	89.18	95.36	71.11	45.50	68.73	71.44
5	50.62	23.00	66.07	41.33	72.48	91.75	96.03	75.56	45.48	68.17	71.38
8	50.29	22.96	66.59	38.67	73.46	92.27	94.70	75.56	45.23	68.94	71.38
20	51.01	24.30	67.77	40.00	75.18	91.75	98.01	75.56	45.09	68.53	71.00
50	51.08	24.76	69.39	40.00	75.92	91.75	97.35	75.56	44.78	67.81	69.97
80	50.42	24.15	68.83	37.33	74.20	92.27	97.35	71.11	44.61	68.22	69.37
100	50.23	23.99	69.80	36.00	74.45	92.27	97.35	71.11	44.57	68.07	69.75

Conclusion

- RAG 기반 언어모델의 디코딩 효율을 근본적으로 재설계한 프레임워크, REFRAG
- **RAG 컨텍스트 내의 희소성(sparsity)**과 **문단 간 약한 attention 구조**를 활용해, 불필요한 연산을 제거하면서 **메모리 사용량과 추론 지연(latency)**을 크게 줄임
- **기존 LLaMA 디코더를 그대로 유지한 채** 압축(compression)–감지(sensing)–확장(expansion)의 세 단계를 통해 효율적인 디코딩을 구현
- **LLaMA-2-7B 기준 최대 30.85배의 속도 향상**, **이전 SOTA 모델(CEPE) 대비 3.75배 빠른 TTFT**, 그리고 **Perplexity 손실 없이 16배 더 긴 컨텍스트 처리 능력**을 달성
- **RAG, 멀티턴 대화, 긴 문서 요약** 등 다양한 long-context 응용에서 모두 검증
- 정확도 손실 없는 초고속 RAG 디코딩